

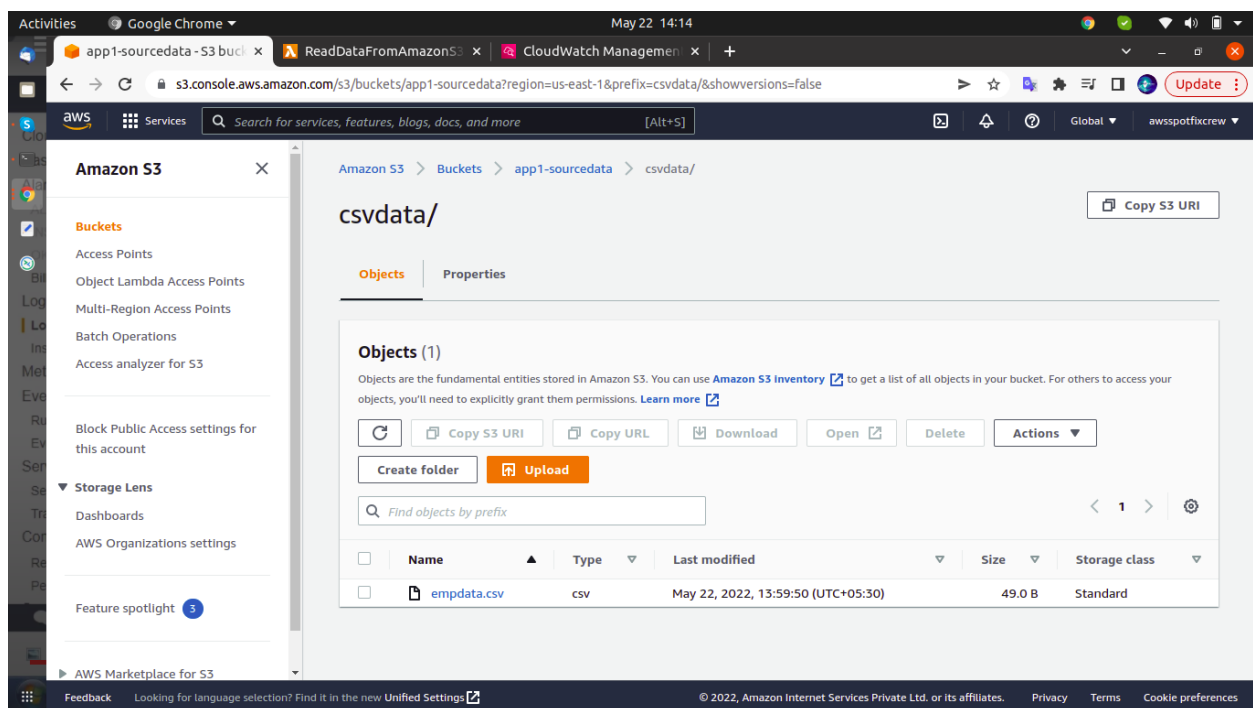
Read s3 CSV file & insert into Amazon Aurora serverless mysql RDS using Python Lambda Function

1) Create s3 bucket

Aws s3 > buckets > create bucket > give a bucket name & select the region as preferred > create bucket

If you want to create a folder inside the bucket

Select the bucket > choose create folder (here i have created a folder inside the bucket named csvdata)



Img: s3 bucket creation

2) Create a Lambda role which has permission to work with s3 and Lambda To create an iam role,

Services > IAM > role > create role > choose use case as Lambda > add s3 full access, cloudwatch full access, RDS full access & VPC full access.

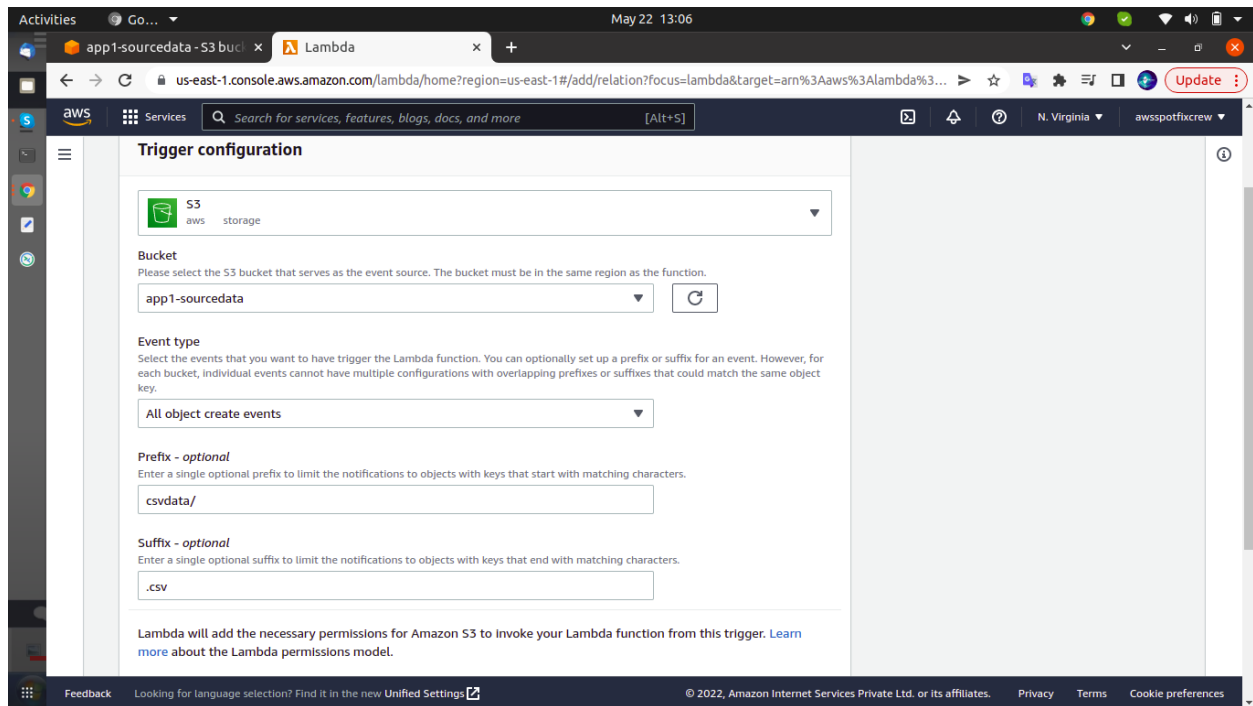
3) Create Lambda function to read the csv file & add s3 as a trigger (need to write the logic to read the csv file)

Services > lambda > functions > create function > choose runtime as python > on the

permission section select the role which we created earlier > on the advanced settings enable VPC & choose VPC, subnets & security group > create function

To add s3 as a trigger (whenever a file uploads to s3, Lambda function is going to get triggered)

Add trigger > choose s3 > select bucket > use “prefix” as the folder created inside the bucket (in my case it is named as csvdata) > use “suffix” as .csv > Add



Img: trigger configuration

To read data from s3 we need to make some changes in the code (Lambda function)

Code:

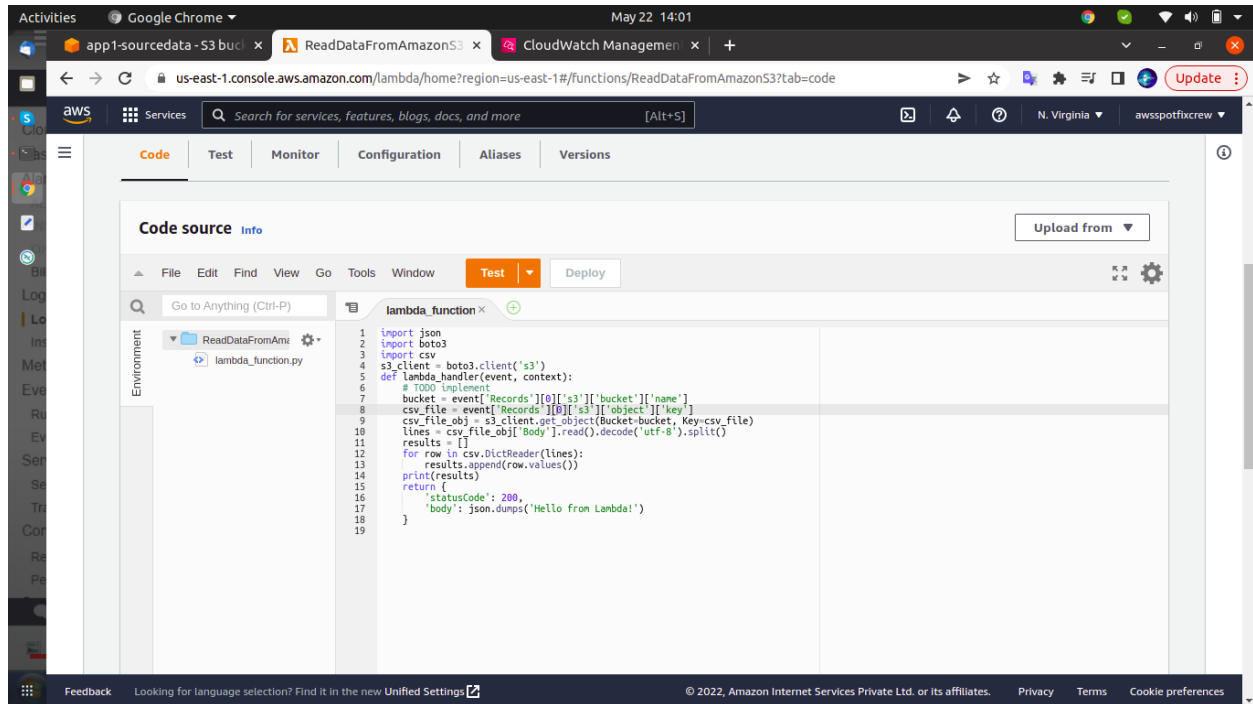
```
import json
import boto3
import csv

s3_client = boto3.client('s3')
def lambda_handler(event, context):
    # TODO implement
    bucket = event['Records'][0]['s3']['bucket']['name']
    csv_file = event['Records'][0]['s3']['object']['key']
    csv_file_obj = s3_client.get_object(Bucket=bucket, Key=csv_file)
    lines = csv_file_obj['Body'].read().decode('utf-8').split()
```

```

results = []
for row in csv.DictReader(lines):
    results.append(row.values())
print(results)
return {
    'statusCode': 200,
    'body': json.dumps('Hello from Lambda!')
}

```

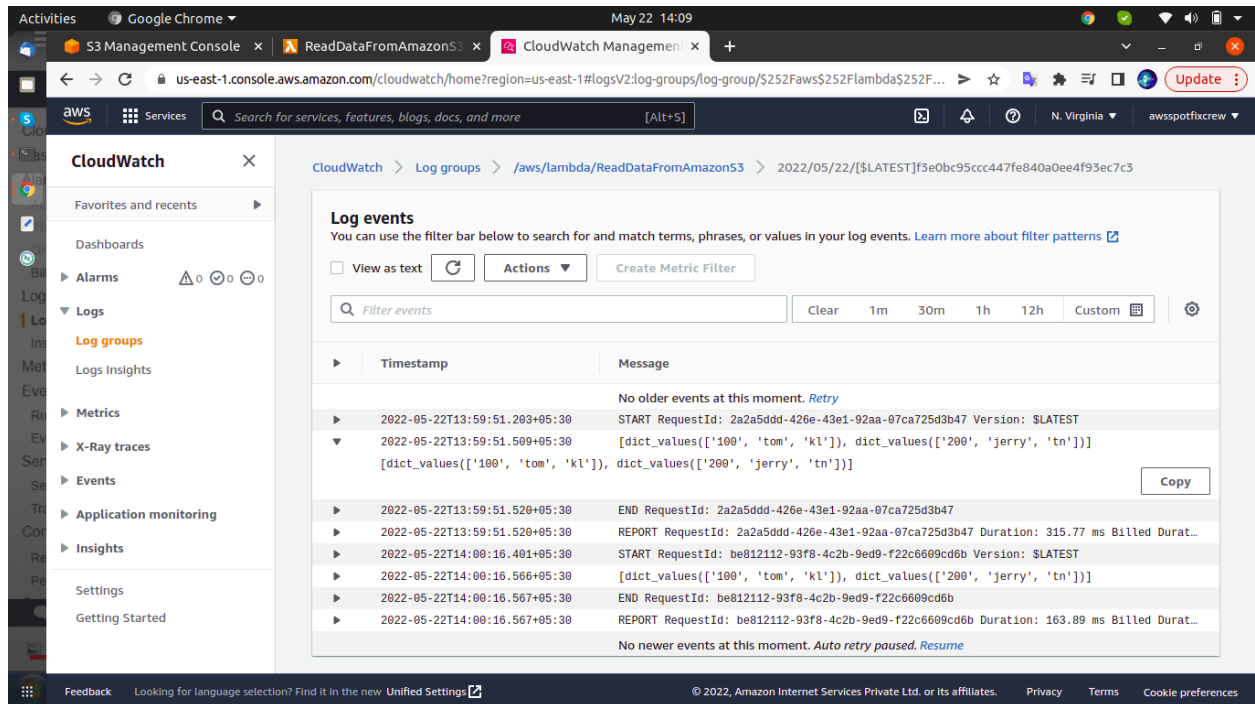


Img: Lambda function to read CSV file from s3 bucket

To check the code we can upload a csv file to the s3 bucket and check the cloudwatch log Stream

To check the cloud watch log stream

Cloudwatch > log group > select the latest log stream



Img: Cloudwatch Log events

Create, connect, & interact with a remote mysql database using amazon rds & python

4) Creating an Amazon Aurora serverlessRDS instance

RDS > create database > choose standard create > select amazon-aurora > select the db instance size as > for edition choose amazon-aurora mysql-compatible edition > for version select aurora (mysql) -5.6.10a > for template choose production > give a specific name to the db cluster > also create the login id (master username & password) as needed > choose serverless for instance configuration > for connectivity select vpc, subnet group & security group > Choose additional configuration & enable the data api > create database

To test the connectivity of the RDS instance; Connect the instance with a mysql client (Create an EC2 instance with the same vpc selected for RDS)

```
# mysql -h app1db.cvp7wqvp14.us-east-1.rds.amazonaws.com -u admin -p
Enter the password : *****
```

After login to the mysql console, Create a database

```
# create database database_name;
```

Also create a table inside the database;

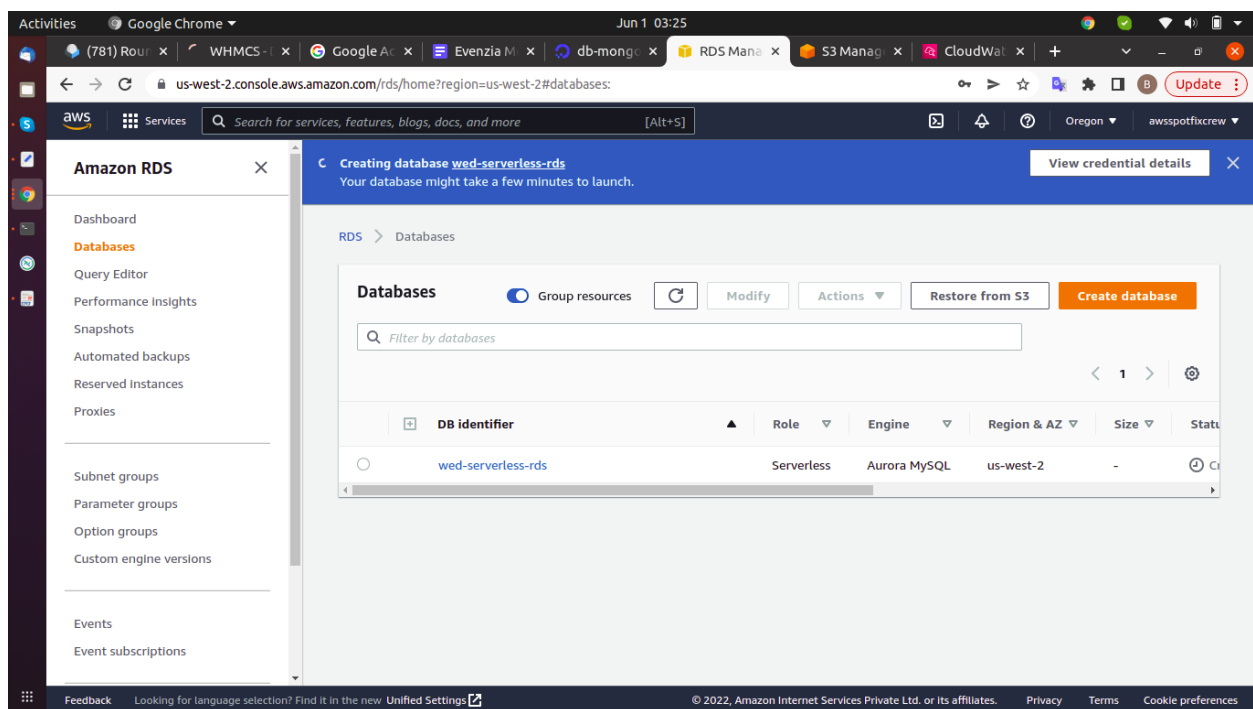
```
# use database_name;
```

```
# create table employee(empid int,empname varchar(40),empaddress varchar(40));
```

To show the table contents;

```
# show tables;
```

```
# select * from table_name;
```



Img: RDS instance

Also make sure that the database is accessible publicly

Connecting to RDS using Lambda function

5) Creating Lambda function to connect to RDS

Lambda > functions > create function > give the function a name > select runtime as python > for execution role choose use an existing role (role we have created earlier on step 2) > select advanced settings & enable VPC then choose VPC, subnets & security group > create

Function

We need to add the module “mysql.connector” to connect the Lambda function with RDS

To do so, we need to understand that when a layer zip is archive is loaded into AWS lambda, it is unzipped to the build/python/lib/python3.9/site-packages folder. For this python lambda function to import the libraries contained in the layer , the libraries should be placed under the python sub-directory of the build/python/lib/python3.9/site-packages folder.

Step1: Create a directory on your local system (eg: aws-lambda-layer-python)

```
# mkdir aws-lambda-layer-python
```

Step2: inside the aws-lambda-layer-python create build/python/lib/python3.9/site-packages

```
# cd aws-lambda-layer-python
```

```
# mkdir -p build/python/lib/python3.9/site-packages
```

Step3: install mysql connector module on the directory build/python/lib/python3.9/site-package

```
# pip install mysql-connector -t build/python/lib/python3.9/site-packages
```

Step4: move on to build directory & create zip file of python directory

```
# cd build/
```

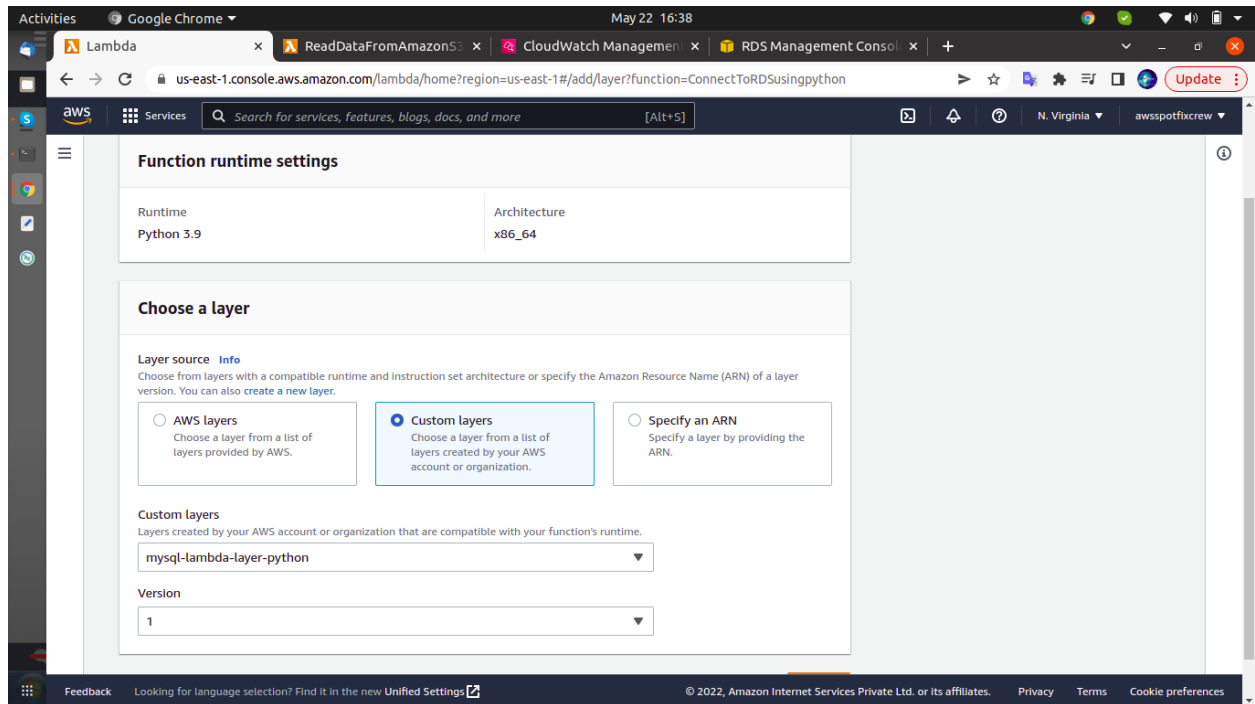
```
# zip -r python.zip python
```

On AWS ,

Go to Lambda > layers > create layer > choose a name for the layer as you like > select upload a zip file > upload the zip file from the build directory > choose runtime as python 3.9 > create.

Add the lambda layer to the existing function,

Go to Lambda > function > select the function created for RDS > layers > add a layer > choose custom layer > select the layer > choose version as 1 > add.



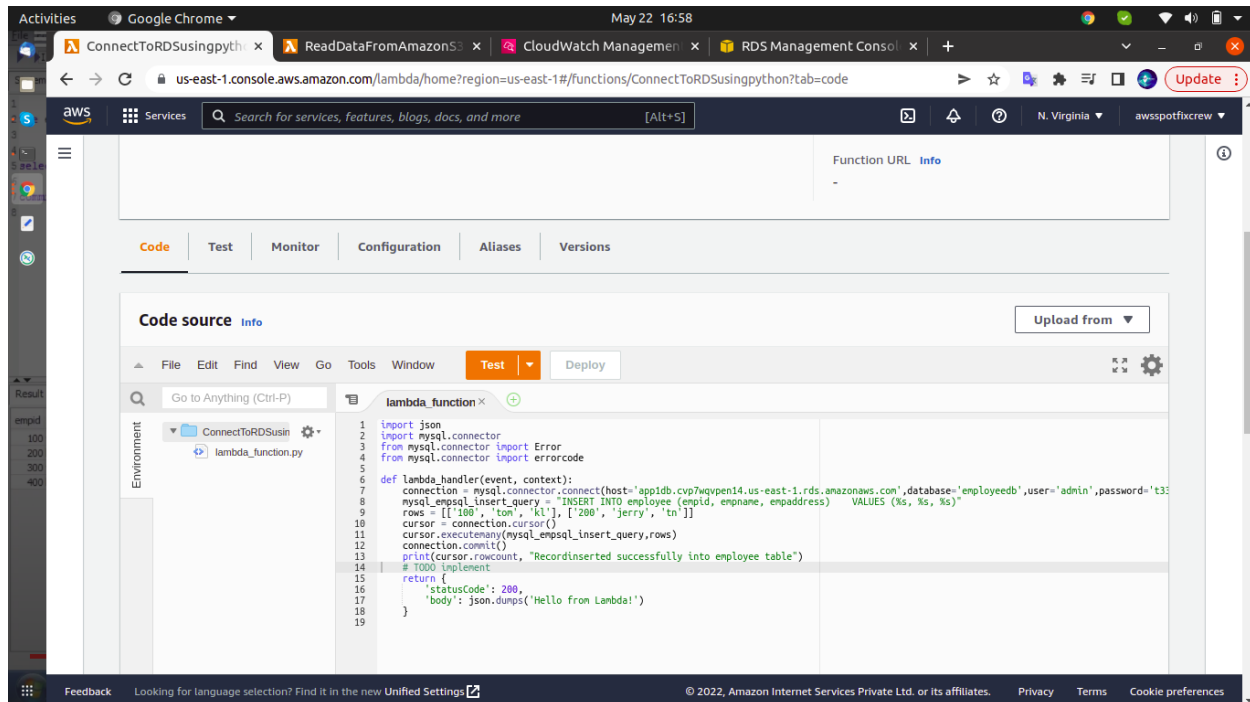
Img: adding layer to a Lambda function

Modify the code to connect our Lambda function to mysql

Code:

```
import json
import mysql.connector
from mysql.connector import Error
from mysql.connector import errorcode

def lambda_handler(event, context):
    connection =
mysql.connector.connect(host='wed-serverless.cluster-cnswkgvolzwl.us-east-2.rds.amazonaws.
com',database='demo',user='admin',password='t33th123!')
    mysql_empsql_insert_query = "INSERT INTO employee (empid, empname, empaddress)
VALUES (%s, %s, %s)"
    rows = [['11', 'sagar', 'cherthala'], ['22', 'alias', 'kochi']]
    cursor = connection.cursor()
    cursor.executemany(mysql_empsql_insert_query,rows)
    connection.commit()
    print(cursor.rowcount, "Record inserted successfully into employee table")
```



Img: Lambda function to connect to RDS

We can check entry of data to the database by using mysql client but in order to read the files from s3 bucket we need to make some changes in the existing Lambda function

6) Reading files from s3 and writing data to RDS (integrating both the lambda functions ie; read data from s3 & connect to RDS)

Add the Lambda layer to the lambda function to read data from s3 bucket

Go to Lambda > functions > select the lambda function (Read data from s3) > select layers > add layer > choose custom layer > select mysql lambda layer which we uploaded earlier > choose version as 1 > add

7) Integrating the code from RDS lambda function to read data from s3 lambda function

(Modules which we need. Add it to the input statement section of s3 Lambda function)

```
import mysql.connector
from mysql.connector import Error
from mysql.connector import errorcode
```

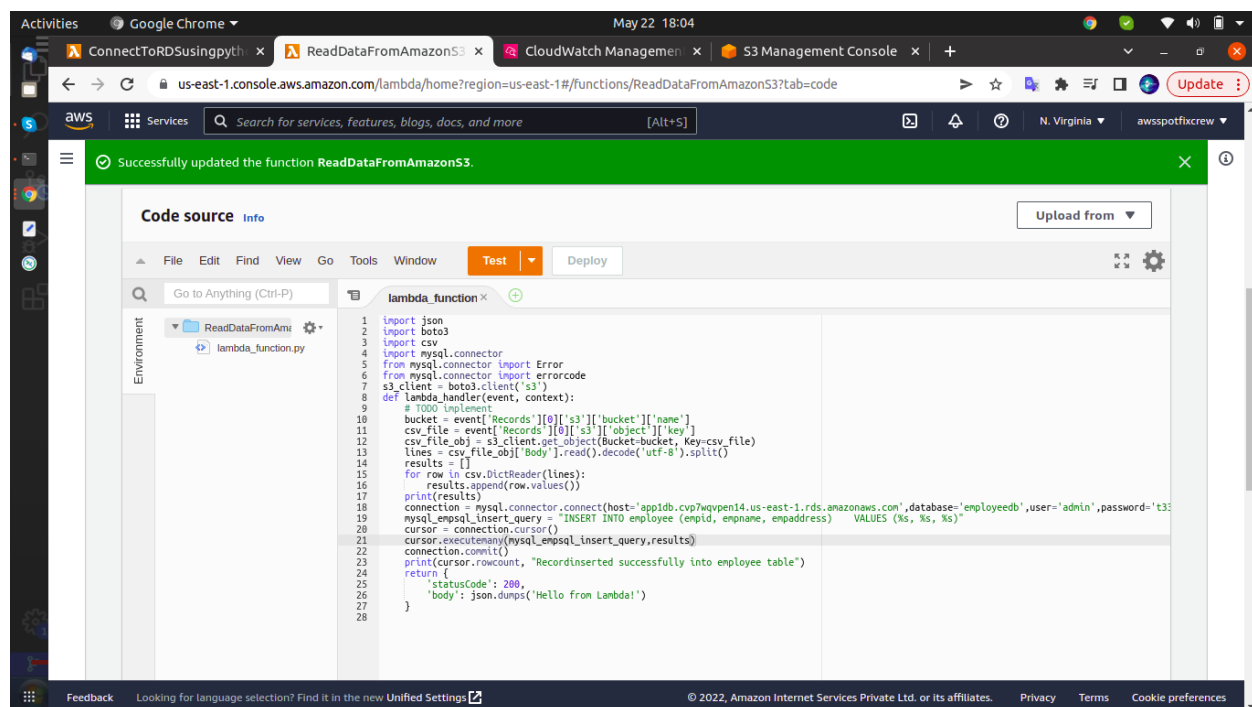
(In order to create the connection & query add these details to the s3 Lambda function)

```
connection =
```

```
mysql.connector.connect(host='app1db.cvp7wqypen14.us-east-1.rds.amazonaws.com',database='employeeedb',user='admin',password='t33th123!')
mysql_emsqsl_insert_query = "INSERT INTO employee (empid, empname, empaddress)
VALUES (%s, %s, %s)"
```

(also copy this logic that reads the csv file to s3 lambda function)

```
cursor = connection.cursor()
cursor.executemany(mysql_emsqsl_insert_query,rows)
connection.commit()
print(cursor.rowcount, "Recordinserted successfully into employee table")
```



Img: integrating code from RDS Lambda to s3 Lambda function

Code:

```
import json
import boto3
import csv
import mysql.connector
from mysql.connector import Error
from mysql.connector import errorcode
s3_client = boto3.client('s3')
def lambda_handler(event, context):
    # TODO implement
```

```

bucket = event['Records'][0]['s3']['bucket']['name']
csv_file = event['Records'][0]['s3']['object']['key']
csv_file_obj = s3_client.get_object(Bucket=bucket, Key=csv_file)
lines = csv_file_obj['Body'].read().decode('utf-8').split()
results = []
for row in csv.DictReader(lines):
    results.append(row.values())
print(results)
connection =
mysql.connector.connect(host='app1db.cvp7wqypen14.us-east-1.rds.amazonaws.com',database='employee',user='admin',password='t33th123!')
mysql_emoji_insert_query = "INSERT INTO employee (empid, empname, empaddress)
VALUES (%s, %s, %s)"
cursor = connection.cursor()
cursor.executemany(mysql_emoji_insert_query,results)
connection.commit()
print(cursor.rowcount, "Record inserted successfully into employee table")
return {
'statusCode': 200,
'body': json.dumps('Hello from Lambda!')}
}

```

Whenever we upload database csv file to amazon s3 bucket we are able to see those records on the table named 'employee'